Focus	Determines what needs to be updated in the real DOM.	Determines how and when updates are applied to the DOM.
Granularity	Operates at the component and element tree level.	Breaks work into small units (fibers) for fine-grained control.
Update style	Applies batched, synchronous updates (pre-Fiber React).	Enables asynchronous, interruptible rendering for better responsiveness.

Now that we have covered Virtual DOM and Fiber Architecture, it is time to dive into more probing questions.

Probing question (applied skill): "What scenarios can trigger re-renders in React, and how can excessive re-rendering affect performance?"

Answer: Re-renders are an essential part of React's component lifecycle, keeping the UI synchronized with application state. However, excessive or unnecessary re-renders can hurt performance by causing redundant DOM updates and re-running component logic. It is critical to understand what triggers re-renders:

- *State changes*—When a component's state is updated using `setState` or a state updater function, it triggers a re-render.
- *Props changes*—While props changes alone do not cause re-renders, they often reflect state changes in a parent component, which does trigger re-rendering of child components.
- *Parent component re-renders*—When a parent component re-renders, it causes all of its child components to re-render as well, regardless of whether their props have changed.
- *Context changes*—If a component consumes a React context and the value of that context changes, it will trigger a re-render of the component.
- *Hook changes*—Changes in a hook, particularly those that manage state or side effects, can cause re-renders.
- *Key prop changes*—Changing the key prop of a component will cause